LeetCode Deep Dive

Expected Time vs. Worst Case: Randomized Selection and Hashing in Practice

Week 15 Supplement – TOAE209/TOAH209: Design and Analysis of Algorithms

Foreign Trade University

Outline

Why walk through LeetCode problems?

- This week's `LEETCODE.md` list has eight Medium problems and **no** official Hard – the same situation as Week 4. This deck picks the best-fitting Medium and adds one well-known, correctly-rated Hard as a stretch problem, exactly as Week 3 did with *Critical Connections in a Network*.
- We pick one **Medium** (*Kth Largest Element in an Array*, via randomized Quickselect) and one **Hard** (*Longest Duplicate Substring*, via binary search + Rabin–Karp rolling hash) – both built directly from this week's lecture: **randomized partitioning** and **universal hashing**.
- The crucial new distinction this week teaches, and that both problems will force you to be precise about: **expected** time vs. **worst-case** time are *not* the same guarantee.

The four-step process we will repeat twice

1. **First instinct.** Write down the algorithm that follows most directly from re-reading the problem statement – usually "sort everything" or "compare every pair."
2. **Measure why it hurts.** Compute the actual complexity, plug in the problem's constraints, and see the number blow up – or see that it does *more work than the question actually asked for*.
3. **Find the structural insight.** A randomized choice (a random pivot, a random-looking hash) that lets you avoid the wasted work *in expectation*, even though no single run is guaranteed to avoid it.
4. **Implement the optimal algorithm**, trace it by hand on a small example, then look for the *programming-level* tricks that make the implementation itself correct and fast.

This mirrors the course

Steps 1–3 are exactly this week's "Las Vegas gambles with time, Monte Carlo gambles with correctness" framing; step 4 is what turns a correct algorithm into working code.

Problem statement

LeetCode 215 – Kth Largest Element in an Array (Medium)

Given an integer array `nums` and an integer k, return the k-th **largest** element (not the k-th distinct element).

- Constraints (LeetCode): $1 \leq k \leq$ `nums.length` $\leq 10^5$, $-10^4 \leq$ `nums[i]` $\leq 10^4$.
- The k-th largest of n elements is exactly the $(n-k)$-th smallest in 0-indexed sorted order – **this week's Quickselect**, computing one order statistic instead of a full sort.

Worked example

`nums = [3,2,1,5,6,4]`, $k = 2$ (2nd largest).

- Sorted ascending: $[1, 2, 3, 4, 5, 6]$; sorted descending: $[6, 5, 4, 3, 2, 1]$.
- The 2nd largest is 5 – equivalently, the element at 0-indexed position $n - k = 6 - 2 = 4$ in ascending sorted order (value 5, at index 4).
- **Answer:** 5.

index (asc. sorted)	0	1	2	3	**4**	5
value	1	2	3	4	**5**	6

First instinct: sort everything

Most direct reading of the problem

"Return the k-th largest" $\Rightarrow$ sort `nums`, then read off `nums[n-k]` (ascending) or `nums[k-1]` (descending).

1: **function** KthLargestNaive(nums, k)
2: sort `nums` in ascending order
3: **return** `nums`$[\, n - k \,]$
4: **end function**

Cost: $O(n \log n)$ – correct, and often exactly what a candidate writes first.

Why this is wasteful (not wrong, wasteful)

- Sorting computes *every* order statistic – the 1st largest, 2nd largest, ..., n-th largest – when the problem only asked for *one* of them.
- A min-heap of size k improves this to $O(n \log k)$ (push each element, pop when size $> k$, answer is the heap's minimum) – better, but still pays a log factor for information you don't need.
- Both approaches impose a total order on data where the question only needs "one specific rank."

The smell to notice

Whenever a problem asks for *one* order statistic (a median, a k-th smallest/largest), full sorting is doing $\log n$ times more work than necessary. That gap is exactly what randomized Quickselect closes.

The structural insight

- Partitioning around a pivot (this week's lecture) tells you, in one pass, *which side* the answer is on – without needing to know the relative order of everything on that side.
- So recurse into **only one** side (unlike QuickSort, which recurses into both), and a **random** pivot keeps the expected shrink factor bounded on *every* input, with no adversarial worst case for a fixed pivot rule.
- This week's Theorem (Quickselect) proved this gives **expected** $O(n)$ – linear, not $O(n \log n)$, because you stop paying for the side you never recurse into.

Optimal algorithm: partition step (Lomuto partition)

```
1: function PARTITION(A, left, right, pivotIdx)
2:     swap A[pivotIdx] and A[right]; pivot ← A[right]
3:     store ← left
4:     for i ← left to right−1 do
5:         if A[i] < pivot then swap A[i], A[store]; store ← store+1
6:         end if
7:     end for
8:     swap A[store], A[right]; return store
9: end function
```

Partitions A[left..right] around the chosen pivot in place: everything $<$ pivot ends up left of `store`, the pivot lands exactly at `store`.

Optimal algorithm: the selection driver

```
1: function SELECT(A, left, right, target)
2:     if left = right then return A[left]
3:     end if
4:     pivotIdx ← random integer in [left, right]
5:     store ← PARTITION(A, left, right, pivotIdx)
6:     if target = store then return A[store]
7:     else if target < store then return SELECT(A, left, store − 1, target)
8:     else return SELECT(A, store + 1, right, target)
9:     end if
10: end function
```

Expected $O(n)$ time – this week's bound, with target $= n - k$. Only **one** recursive call ever happens (unlike QuickSort's two).

Trace on the worked example

`nums = [3,2,1,5,6,4]`, target $= n - k = 4$. (One concrete run; any pivot sequence gives the same final answer.)

call	array state before	pivot chosen	result
Select(0,5,4)	$[3, 2, 1, 5, 6, 4]$	index 5 (value 4)	partition $\rightarrow$ $[3, 2, 1, 4, 6, 5]$, pivot lands at index 3; target $4 > 3$
Select(4,5,4)	$[6, 5]$ (indices $4, 5$)	index 5 (value 5)	partition $\rightarrow$ $[5, 6]$ at indices $4, 5$; pivot lands at index 4; target $4 = 4$: **return** $A[4] = 5$

Answer

Two recursive calls, one side discarded each time – returns 5, matching the 2nd-largest element. Notice the first call never looked at the relative order of indices $0, 1, 2$ once it knew the answer was on the other side.

Complexity: naive vs. optimal

Approach	Time	Guarantee
Sort everything	$O(n \log n)$	worst-case
Min-heap of size k	$O(n \log k)$	worst-case
Randomized Quickselect	$\boldsymbol{O(n)}$	**expected** – worst-case $O(n^2)$
Deterministic (median-of-medians)	$O(n)$	worst-case, but larger constants

The precise claim, stated carefully

Randomized Quickselect is expected-linear on *every* input, with no adversarial case for a fixed pivot rule – but any *single* run can, with tiny probability, still take $\Theta(n^2)$. This week's advances deck covers the fully deterministic worst-case-$O(n)$ alternative (Blum–Floyd–Pratt–Rivest–Tarjan, 1973) if you need a hard guarantee instead of an expected one.

Python implementation (part 1: partition)

```python
import random

def findKthLargest(nums, k):
    target = len(nums) - k           # 0-indexed rank of the k-th largest

    def partition(left, right, pivot_index):
        pivot = nums[pivot_index]
        nums[pivot_index], nums[right] = nums[right], nums[pivot_index]
        store = left
        for i in range(left, right):
            if nums[i] < pivot:
                nums[i], nums[store] = nums[store], nums[i]
                store += 1
        nums[right], nums[store] = nums[store], nums[right]
        return store
```

Python implementation (part 2: select)

```
    def select(left, right):
        if left == right:
            return nums[left]
        pivot_index = random.randint(left, right)
        store = partition(left, right, pivot_index)
        if target == store:
            return nums[store]
        if target < store:
            return select(left, store - 1)
        return select(store + 1, right)

    return select(0, len(nums) - 1)
```

`select` is nested inside `findKthLargest` purely so it can close over `nums` and `target` without passing them explicitly each call.

Implementation tips & tricks (the programming side)

- **Select is tail-recursive – turn it into a loop.** Unlike QuickSort, Quickselect only ever recurses into *one* side. That means the recursive call can be replaced by simply updating `left`/`right` in a `while` loop, eliminating recursion depth entirely – important since an adversarial (if unlucky) sequence of pivot draws can still recurse $\Theta(n)$ deep, risking a `RecursionError` at n near Python's default limit.
- **Partition in place, not with new lists.** The lecture's proof-friendly pseudocode builds new lists $L = \{y < x\}$, $R = \{y > x\}$; a real implementation partitions the *same* array in place (as above) to avoid $O(n)$ allocations per recursive call.
- **`random.randint`, not a hand-rolled generator.** Python's default `random` module (Mersenne Twister) is fast and statistically good enough here; there is no reason to reach for `secrets` (cryptographic randomness), which is slower and solves a different problem.

Problem statement

LeetCode 1044 – Longest Duplicate Substring (Hard)

Given a string s, return any substring that occurs at least twice in s (occurrences may overlap) with the **longest** possible length. Return `""` if no such substring exists.

- Constraints (LeetCode): $2 \leq |s| \leq 3 \times 10^4$, lowercase English letters only.
- This is a **static** question (unlike Problem 1's *online* order statistic) about a whole string – and it is naturally attacked with this week's **hashing** section, not partitioning.

Worked example

`s = "banana"` ($n = 6$).

index	0	1	2	3	4	5
char	b	a	n	a	n	a

- Substring `"ana"` occurs at index 1 (`s[1:4]`) **and** index 3 (`s[3:6]`) – an *overlapping* duplicate.
- No length-4 substring repeats (`"bana"`, `"anan"`, `"nana"` are all distinct).
- **Answer:** `"ana"` (length 3).

First instinct: compare every pair of substrings

Most direct reading of the problem

For each candidate length L from $n-1$ down to 1, look at every substring of length L and check whether it matches any other substring of the same length.

1: **for** $L \leftarrow n-1$ **downto** 1 **do**
2: **for** each pair of start indices $i < j$ with $j + L \leq n$ **do**
3: **if** $s[i..i+L) = s[j..j+L)$ **then return** $s[i..i+L)$
4: **end if**
5: **end for**
6: **end for**

Why this is bad

- For a fixed L, comparing all $O(n)$ pairs of length-L substrings costs $O(n)$ comparisons $\times$ $O(L)$ per comparison $= O(nL)$.
- Summed over all n possible lengths L: $O(n^2 L_{\text{avg}}) = O(n^3)$ in the worst case.
- Plug in the constraint $n = 3 \times 10^4$: $n^3 = 2.7 \times 10^{13}$ – utterly infeasible, even though the problem "only" asks about a string of length 30,000.

The smell to notice

"Compare every substring against every other substring" re-derives string equality from scratch each time, throwing away the fact that most substrings share huge overlapping prefixes with their neighbors. That redundancy is exactly what a **rolling hash** removes.

The structural insight

- **Monotonicity.** If a duplicated substring of length L exists, one of length $L-1$ also exists (trim it) – so "does a duplicate of length $\geq L$ exist" is a monotone yes/no question in L, and **binary search** finds the largest feasible L in $O(\log n)$ probes instead of trying every length.
- **Rolling hash (Rabin–Karp), this week's universal-hashing idea.** Treat each length-L window as a base-26 number mod a large modulus; sliding the window by one position updates the hash in $O(1)$ instead of rehashing $O(L)$ characters – so one feasibility check, for a fixed L, costs $O(n)$ total.
- Combined: $O(\log n)$ binary-search steps $\times$ $O(n)$ per step $= O(n \log n)$ – versus the naive $O(n^3)$.

Trace on the worked example

`s = "banana"`, $n = 6$. Binary search L over $[1, n-1] = [1, 5]$.

iteration	mid L	feasibility check
1	3	substrings `ban,ana,nan,ana`: index 3's `"ana"` repeats index 1's $\Rightarrow$ **feasible**. Record `"ana"`; search $L > 3$.
2	4	substrings `bana,anan,nana`: all distinct $\Rightarrow$ **infeasible**; search $L < 4$.
–	–	range exhausted (lo$=4 >$hi$=3$): stop

Answer

Two feasibility checks (not five, one per length) find the answer `"ana"` – matching the expected output.

Complexity: naive vs. optimal

Approach	Time	At $n = 3 \times 10^4$
Compare every substring pair, every length	$O(n^3)$	$\sim 2.7 \times 10^{13}$ ops
Binary search + rolling hash	$\boldsymbol{O(n \log n)}$	$\sim 4.5 \times 10^5$ ops

The precise claim, stated carefully

A hash *match* is a candidate, not a proof – two different substrings can collide. The implementation below always verifies with a direct string comparison on a hash hit, so the algorithm is correct with certainty (Las Vegas in spirit), not merely "probably correct" (Monte Carlo).

Python implementation (part 1: the feasibility check)

```python
def longestDupSubstring(s):
    n = len(s)
    nums = [ord(c) - ord('a') for c in s]
    base, mod = 26, (1 << 32) - 1

    def search(L):
        h = 0
        for i in range(L):
            h = (h * base + nums[i]) % mod
        seen = {h: [0]}
        power = pow(base, L - 1, mod)          # precomputed once per call
        for start in range(1, n - L + 1):
            h = ((h - nums[start - 1] * power) * base
                 + nums[start + L - 1]) % mod
            if h in seen:
                for j in seen[h]:
                    if s[j:j + L] == s[start:start + L]:
                        return start            # verified, not just hashed
                seen[h].append(start)
            else:
                seen[h] = [start]
        return -1
```

Python implementation (part 2: the binary search driver)

```
    lo, hi, result = 1, n - 1, ""
    while lo <= hi:
        mid = (lo + hi) // 2
        start = search(mid)
        if start != -1:
            result = s[start:start + mid]
            lo = mid + 1                        # duplicate found: try longer
        else:
            hi = mid - 1                        # none found: try shorter
    return result
```

`hi = n - 1`, not `n`: a duplicate needs two *distinct* start positions, so the longest possible duplicate has length $n - 1$.

Implementation tips & tricks (the programming side)

- **Precompute the power once per length, not once per position.** `power = pow(base, L-1, mod)` costs $O(\log L)$ via fast modular exponentiation and is reused across all $n - L$ rolling updates – computing it fresh inside the loop would silently reintroduce an $O(n \log L)$ factor.
- **Only slice strings on a hash hit.** `s[j:j+L]` is $O(L)$; calling it unconditionally on every position would erase the entire benefit of rolling hashes. It is called only when two hashes already match – rare, so the amortized cost stays low.
- **A fixed modulus like $2^{32} - 1$ is convenient but not adversary-proof.** Because it is a well-known constant, a maliciously constructed test string could in principle be built to collide against it. This week's own **universal hashing** idea (choose the hash function's parameters at random, or use a large random prime modulus / double hashing) removes that risk – exactly the same fix that motivated universal hashing in lecture.

Summary: two problems, one lesson

1. **Kth Largest Element** needs only *one* order statistic – randomized Quickselect answers it in expected $O(n)$ by recursing into only the side that matters, instead of sorting (or heap-ordering) everything.
2. **Longest Duplicate Substring** needs a *global* yes/no answer about a whole string at many candidate lengths – monotonicity turns that into a binary search, and a rolling hash (this week's universal-hashing idea) makes each check $O(n)$ instead of $O(n^2)$.
3. Both times, the naive instinct did *more work than the question asked for* (sorting everything; comparing every substring pair); both times, randomness – a random pivot, a random-looking hash – let the algorithm skip the unnecessary work *in expectation*.
4. Both times, precision about the guarantee mattered: **expected** $O(n)$ for Quickselect (worst case $O(n^2)$, unless you pay median-of-medians' constants), and a rolling hash that is correct *with certainty* only because every match is verified before being trusted.

This closes out the course's LeetCode-walkthrough series

`LEETCODE.md` lists seven more Week 15 problems (*Shuffle an Array*, *Random Pick with Weight*, *Insert Delete GetRandom O(1)*, ...) – and fourteen weeks of problems before them. The same four-step process applies to all of them: find the naive approach, measure why it hurts, find the structural insight, then mind the implementation details.